

Assembly Language Programming

Study Guide

Mrs. Ranita Sen
Assistant Professor
Dept of ECE, PIET
Parul University

1. Fundamentals of Assembly Language	1
2. Program structure	2
3. Directives	3
4. Programming exercises	4
5. Stack and Subroutines	6

3.1 Fundamentals of Assembly Language

- Assembly Language is a low-level programming language that is closely related to machine code. It is a human-readable representation of machine code. Unlike high-level languages like Python or Java, assembly language is closely tied to the architecture of a specific computer processor.
- Assembly language acts as a human-readable version of machine language. Each instruction in assembly corresponds to a single machine instruction, making it extremely powerful and efficient—but also harder to learn and write.

Data Transfer Instructions: These instructions move data between registers, memory, and I/O devices without altering the data.

Examples:

- MOV (Move data between registers)
- MVI (Move immediate data)
- LDA (Load accumulator directly from memory)
- STA (Store accumulator directly into memory)
- LDAX / STAX (Load/Store accumulator indirect)
- IN, OUT (Input/Output operations)
- XCHG (Exchange HL and DE register pair)

Arithmetic Instructions: These perform arithmetic operations like addition, subtraction, increment, and decrement.

Examples:

- ADD, ADI (Add register/immediate to accumulator)
- SUB, SUI (Subtract register/immediate from accumulator)
- INR, DCR (Increment/Decrement register or memory)
- INX, DCX (Increment/Decrement register pair)
- DAD (Add register pair to HL)
- CMP, CPI (Compare with accumulator)

Logical Instructions: These instructions perform bitwise logical operations.

Examples:

- ANA, ANI (AND with accumulator)
- ORA, ORI (OR with accumulator)
- XRA, XRI (XOR with accumulator)
- CMA (Complement accumulator)
- CMC (Complement carry)
- STC (Set carry)
- RLC, RRC, RAL, RAR (Rotate operations)

Branching Instructions: These control the flow of execution by jumping to other memory locations.

Examples:

- JMP, JC, JNC, JZ, JNZ (Unconditional and conditional jumps)
- CALL, CC, CNC, CZ, CNZ (Subroutine calls)
- RET, RC, RNC, RZ, RNZ (Return from subroutine)
- PCHL (Jump to address in HL)

Control Instructions: These control processor operations.

Examples:

- NOP (No operation)
- HLT (Halt the processor)
- DI / EI (Disable/Enable interrupts)
- SIM / RIM (Set/Read interrupt mask)

3.2 Program Structure

- **Start (Origin):** The execution of an 8085 assembly language program begins at a specific memory address. To define this address, the ORG (origin) directive is used. This directive tells the assembler where in memory the program should be loaded and executed from. For instance, if you use ORG 2000H, the assembler will begin placing the machine code of the instructions from memory address 2000H onward. This is particularly useful in embedded systems programming, where memory layout and address control are critical. The ORG directive does not produce any machine code—it simply guides the assembler.
 - Set the starting address using the ORG directive.
- **Initialization:** Once the starting point is set, the next step is initialization. This involves preparing the processor and system for operation. Initialization typically includes loading registers with required values using instructions like MVI, LXI, or MOV. These instructions move data into registers or register pairs. For example, MVI A, 05H loads the accumulator with the value 05H. Initialization may also include setting up memory locations or I/O ports, especially in interfacing scenarios. For example, if an output device is mapped to port 01H, an instruction like OUT 01H may be used after loading the accumulator with a value to send. Proper initialization ensures that all necessary components are in a known and correct state before executing the main logic.
 - Load registers
 - Initialize memory or ports (if needed)
- **Main Program Logic:** This section performs the actual operations the program is designed to execute. It typically involves arithmetic or logical instructions like ADD, SUB, ANA, ORA, etc. Control flow instructions such as JMP, JC, JZ, CALL, and RET are used to create loops, implement conditional branching, and call subroutines. For example, a loop can be created using a decrement-and-jump combination: DCR B followed by JNZ LOOP. Logical flow is critical for decision-making processes, repeating tasks, and structuring larger programs modularly using subroutines. The use of flags (like Zero, Carry) allows conditional instructions to guide the program

based on computation results.

- Perform arithmetic/logical operations
- Use loops, conditions, calls
- **Terminate/End:** At the conclusion of the program, it is necessary to stop the processor from executing unintended memory areas. The HLT (halt) instruction is used for this purpose. When the 8085 executes the HLT instruction, it stops executing further instructions and enters a halted state, awaiting a reset. This ensures controlled program termination. The END directive is also used in this section—not for the processor, but for the assembler. It tells the assembler that the source code has ended and no more instructions need to be processed. Like ORG, the END directive does not generate any machine code; it's purely for the assembler's use.
 - HLT instruction to stop execution
 - END directive for assembler

3.3 Directives

Directives (also known as assembler directives or pseudo instructions) are instructions given to the assembler, not the microprocessor. They help the assembler understand how to process the code but do not generate machine code themselves. Here are the most commonly used directives in 8085 assembly language:

Common 8085 Assembler Directives

Directive	Description
ORG	Sets the starting address for the program. Syntax: ORG 2000H
END	Marks the end of the source code. Tells the assembler to stop assembling.
EQU	Assigns a constant value to a symbol. Syntax: VALUE EQU 10H
DB	Define Byte – used to define 8-bit data values. Syntax: DATA DB 0AH
DW	Define Word – used to define 16-bit data values. Syntax: DATA DW 1234H
DS	Define Storage – reserves space in memory. Syntax: ARRAY DS 10 (reserves 10 bytes)
LABEL	Used to assign a name to a memory location. Syntax: START: MVI A, 01H

- Directives are for the assembler, not for the CPU. They help organize code, allocate memory, and define constants. They make assembly language programming more readable and manageable.
- The ORG directive is used to specify the starting address of the program or a block of code or data. It tells the assembler to begin placing the subsequent machine code or data from the specified memory location. This is essential when writing programs for systems with fixed memory maps or when interfacing with hardware that requires programs to reside at specific addresses. For example, ORG 2000H instructs the assembler that the following code should be stored beginning at memory address 2000H. It does not generate any machine code but helps organize code placement.
- The END directive indicates the end of the assembly source code. It tells the assembler that it should stop reading the instructions from this point onward. This directive is mandatory in most assemblers

to avoid errors or undefined behavior. Even if more lines exist after the END statement, the assembler will ignore them. For example, once the assembler reads END, it stops compiling the source file.

- The EQU directive is used to define a constant value that will be used throughout the program. It allows the programmer to give a name to a fixed value, improving readability and making it easier to update constants later. For instance, the statement `VALUE EQU 10H` assigns the hexadecimal value 10H to the symbol VALUE. Anywhere VALUE appears in the code, it will be replaced by 10H during assembly. This does not reserve memory or generate code—it simply defines symbolic constants.
- The DB directive is used to declare 8-bit data in the program. It tells the assembler to allocate memory and store the specified byte at that location. For example, `DATA DB 0AH` stores the 8-bit value 0AH (decimal 10) in memory. Multiple bytes can be defined in a single line using commas (e.g., `DB 1, 2, 3, 4`). This directive is particularly useful for initializing variables, lookup tables, and strings.
- The DW directive is used to declare 16-bit data (also called a "word" in 8085 terminology). It instructs the assembler to store a 16-bit value at the current memory location. For instance, `DATA DW 1234H` stores the 16-bit value 1234H in two consecutive memory locations, with the lower byte stored first (due to little-endian format). This directive is useful when dealing with address pointers or large numerical data.
- The DS directive is used to reserve memory space without assigning any initial values. It tells the assembler to leave a block of uninitialized bytes in memory. For example, `ARRAY DS 10` reserves 10 bytes of space in memory for later use, which can be used for variables, buffers, or stack. This is particularly useful in embedded applications where memory management is critical.
- A LABEL is a symbolic name assigned to a specific memory location, usually used before an instruction or a data definition. Labels improve program readability and are critical for flow control (e.g., loops, jumps, subroutine calls). For example, `START: MVI A, 01H` assigns the label START to the memory address of the instruction `MVI A, 01H`. Labels are referenced by branch instructions like `JMP START`, making code easier to manage and modify.

3.4 Programming Exercises

Example 1: Add two numbers stored in memory

ORG 8000H	Starting address
LDA 8000H	Load first number to accumulator
MOV B, A	Move to register B
LDA 8001H	Load second number
ADD B	Add contents of B to accumulator
STA 8002H	Store result in 8002H
HLT	Stop execution
END	End of program

Example 2: Subtract two numbers

MVI A, 3CH	A = 3CH
MVI B, 2FH	B = 2FH

SUB B A = A - B = 0DH
HLT

Example 3: Move immediate data to memory (store 8-bit number)

LXI H, 3000H Load HL with address 3000H
MVI M, 55H Store 55H at memory location 3000H
HLT

Example 4: Find the largest number from an array of n numbers

LXI H, 3000H HL points to the first memory location
MOV C, M Store the count at 3000H in C
INX H
MOV A, M Assume first number is largest
DCR C
LOOP: INX H
CMP M Compare A with next number
JC SKIP If A < M, skip
MOV A, M Else, move M to A
SKIP: DCR C
JNZ LOOP
STA 3100H Store the largest at 3100H
HLT

Example 5: Sorting an array in ascending order (bubble sort)

LXI H, 3000H Point to the first memory location (data size in 3000H)
MOV C, M Store count in C
DCR C Outer loop count = N - 1
LOOP1: LXI H, 3001H Start of array (skip count byte)
MOV B, C Inner loop counter
LOOP2: MOV A, M Load current element
INX H
CMP M Compare with next element
JC SKIP If already in order, skip
MOV D, M Swap if not
MOV M, A
DCX H
MOV M, D
INX H
SKIP: DCR B

JNZ LOOP2	Repeat inner loop
DCR C	
JNZ LOOP1	Repeat outer loop
HLT	

Example 6: Multiplication using repeated addition

LXI H, 2000H	
MOV B, M	B = Multiplicand
INX H	
MOV C, M	C = Multiplier
MVI A, 00H	Clear A to store result
LOOP: ADD B	
DCR C	
JNZ LOOP	
STA 3000H	Store result
HLT	

3.5 Stack and Subroutines

A stack is a special area of memory used for temporary storage of data, particularly during subroutine calls, interrupts, and register saves/restores.

Key Points:

- LIFO (Last In, First Out) structure.
- Grows downward in memory (from higher to lower addresses).
- Controlled using the Stack Pointer (SP) register.

Instructions:

- **PUSH rp**: Saves the contents of a register pair onto the stack.
- **POP rp**: Restores data from the stack into a register pair.
- **XTHL**: Exchanges top of stack with HL register pair.

A subroutine is a block of code designed to perform a specific task, which can be called from multiple places in the program.

Subroutine Structure:

- CALL instruction is used to transfer control to the subroutine.
- RET instruction is used to return to the main program. SPHL: Copies HL into SP.

Memory Map:

Address	Code	Description
2000H	LXI H, 2500H	Load address 2500H in HL (result)
2003H	LDA 3000H	Load A with first number
2006H	MOV B, A	Move A to B
2007H	LDA 3001H	Load A with second number
200AH	CALL 2100H	Call subroutine at 2100H
200DH	STA 2500H	Store result in 2500H
2010H	HLT	Stop

Subroutine at 2100H: Adds B and A

2100H: PUSH PSW	Save A and flags
2101H: ADD B	$A = A + B$
2102H: POP PSW	Restore A and flags (if needed)
2103H: RET	Return to main program

- CALL 2100H: Pushes the return address (200DH) onto the stack and jumps to 2100H.
- RET: Pops the return address from the stack and resumes execution.
- PUSH/POP: Stack used to save/restore registers or flags temporarily.

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat
GRADE



<https://paruluniversity.ac.in/>